CSC3002F - Operating Systems II

Scheduling Assignment

Connor Morrison (MRRCON006)

19th May 2026

Introduction

Allegra the barman is an abstraction of the low-level workings of a CPU. The study of CPU scheduling algorithms can be likened much to the analogy of a barman serving drinks at a bar, where each patron (customer) is a process and each of their drink orders is a CPU burst. In this analogy, Allegra the barman acts as the CPU and her method of serving patrons is a scheduling algorithm. This provides us with a real world example of what CPU scheduling algorithms do and what these algorithms aim to achieve.

The CPU scheduling algorithms that will be tested within this analogy are First-Come First-Serve (FCFS), Shortest Job First (SJF), Priority, and Multilevel Feedback Queue (MLFQ). We are provided the information that none of the algorithms use pre-emption at the level of the individual drink order.

We will evaluate the overall success of the algorithms by comparing their performances with respect to the response and waiting times (both per order, and the total per patron), and turnaround time, as well as throughput, predictability, fairness, and possibility of starvation.

The goal of this assignment is to determine the CPU scheduling algorithm that, when applied to the bar setting, is to satisfy the bar owner.

In order to achieve maximum satisfaction, we need to serve as many customers in the shortest time frame (throughput), while at the same time, ensuring that all customers feel fairly treated such that they will, on average, have the best experience. Taking both of these things into account will help maximise nightly profit, while encouraging consequent visits, hopefully providing longer term success to the bar.

Methods

One of the most important steps in the experimental process is creating valid and reliable data in a repeatable way. This process can be broken down into steps to first generate the data, and second validate the data.

Method of data generation

To generate reliable and repeatable data we need to define, in context, what this would mean. For our results to be reliable we must first ensure that we fairly test each algorithm. We can do this by having simulations for each algorithm run with the same `seed`, `sleep_time`, `total_patrons` configuration.

`sleep_time` describes to the Java program the resting time of the barman before processing the next order, which in this scenario is unnecessary, so we set it to 0.

We need a range of different `total_patrons` to test these algorithms under different workloads. The values of 5, 20, and 50 have been chosen for this experiment.

According to the Central Limit Theorem, the minimum number of unique simulations required to model their distributions is 30. Meaning that we will need to test every scheduling algorithm 30 times for each value of `total_patrons`. To do this we can loop through seeds 1 to 30 for each `scheduling_algorithm`, `total_patrons` pairing.

We created a bash script Figure 1 to run through this repeated simulation process.

We need to also ensure that the generated data is in a suitable format for analysing. This was done by completing the `recordCompletedOrder` function in `Barman.java` to print all of the *per-order metrics* that the barman class provided in the format "*patron,drink,priority,queue_level,seq_no,service_start_time,arrival_time,completion_time,waiting_time,imbibing_time,turnaround_time,enqueue_time,execution_time,response_time*." This output prints this for every completed drink order into the error console. We retrieve only the error console output of the application by calling `2>&1 ./dev/null` after the call of the `make` function.

But before we save the metrics in a `.csv` file, we need to attach the individual simulation configuration "`total_patrons`,`schedalg`,`seed`," to the start of each line in the error console. This is done by `sed "s/^/$patrons,$schedalg,$seed,/"`. The final outputs are appended to `simulation.csv`.

By running our bash script, our generation of repeatable and reliable data is complete.

simulation.sh

```
echo
"total_patrons,schedalg,seed,patron,drink,priority,queue_level,seq_no,service_start_time,arrival_time
,completion_time,waiting_time,imbibing_time,turnaround_time,enqueue_time,execution_time,response_time
" > simulation_results.csv
for patrons in 5 20 50
do
    for seed in {1..30}
    do
        for schedalg in 0 1 2 3
        do
            #args format ARGS="patrons scheduling_algorithm 0 seed"
            make --silent run ARGS="$patrons $schedalg 0 $seed" 2>&1 >/dev/null | sed
            "s/^/$patrons,$schedalg,$seed,/" >> simulation_results.csv
        done
    done
done
# Outputs the results into simulation_results.csv in the format:
#
patron,drink,priority,queue_level,seq_no,service_start_time,arrival_time,completion_time,waiting_time
,imbibing_time,turnaround_time,enqueue_time,execution_time,response_time
```

Figure 1: Screenshot of Bash Simulation Script

Method of data validation

The exact code/methods used to obtain results in the section can be found in `MRRCON006 Final Report.qmd`.

To validate the data and code for the analysis of the algorithms, we need to ensure that

(a) the data is complete, **(b)** the data follows the basic CPU scheduling rules, and **(c)** the data produced identical workloads for each algorithm across seeds.

(a) Data completeness

We check that the correct amount of simulations was run and that there were no missing values

`Validation: Seeds per workload/algorithm combination:`

total_patrons	schedalg	seeds_count
5	FCFS	30
5	SJF	30
5	Priority	30
5	MLFQ	30
20	FCFS	30
20	SJF	30

total_patrons	schedalg	seeds_count
20	Priority	30
20	MLFQ	30
50	FCFS	30
50	SJF	30
50	Priority	30
50	MLFQ	30

```
Number of missing values: 0
```

Which correctly matches what we expect to see.

(b) Code and data is logically correct

To test this we check whether all $Turnaround \geq Waiting + Execution$, and $Response \leq Waiting$. If there are 0 logic errors, then that shows that the code and data generated is valid.

```
0 exceptions, Validation Passed: Turnaround times are logically consistent.
```

```
0 exceptions, Validation Passed: Response time is always <= total waiting time.
```

(c) Are the workloads identical across algorithms for the same seed

We tested whether the each algorithm had and identical total and type of drinks for the same seed, total_patrons pairing.

```
Passed! The workload produced was identical across algorithms for a given seed
```

Results and Analysis

Per-Order metrics

What does this mean in terms of the bar and CPU scheduling

Response time

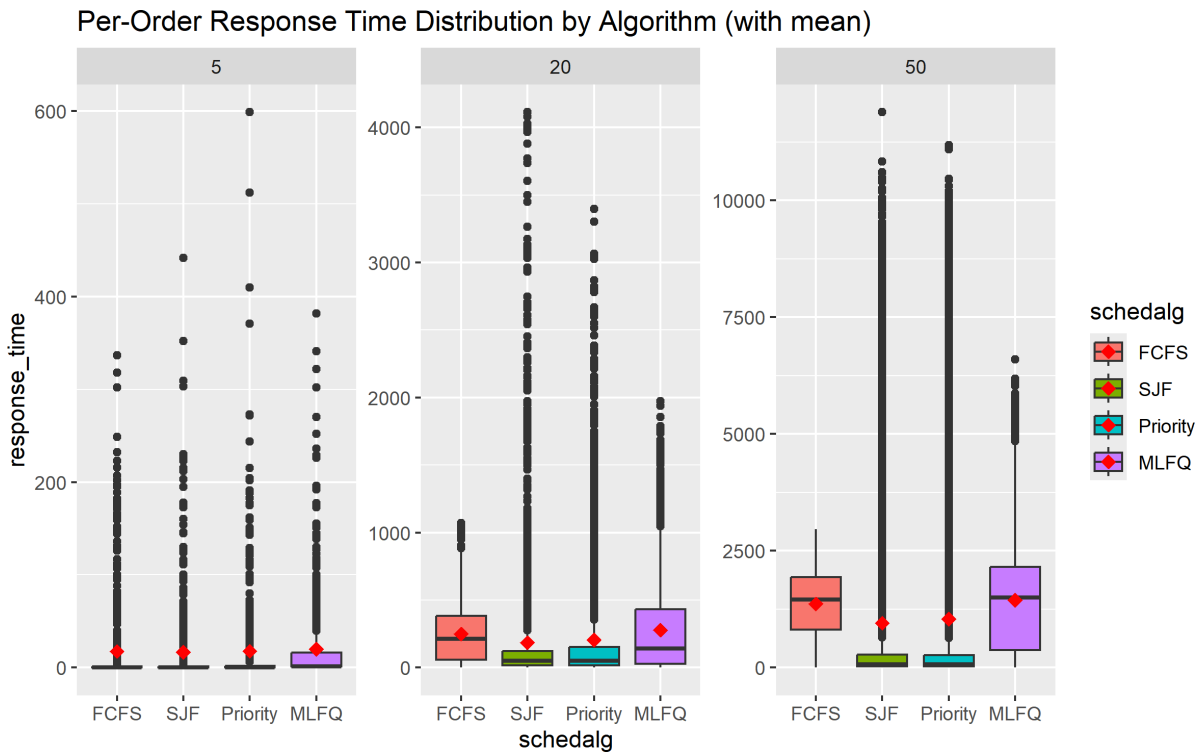

Figure 2: Boxplots of Per-Order Response Times

As seen in Figure 2.

Waiting time

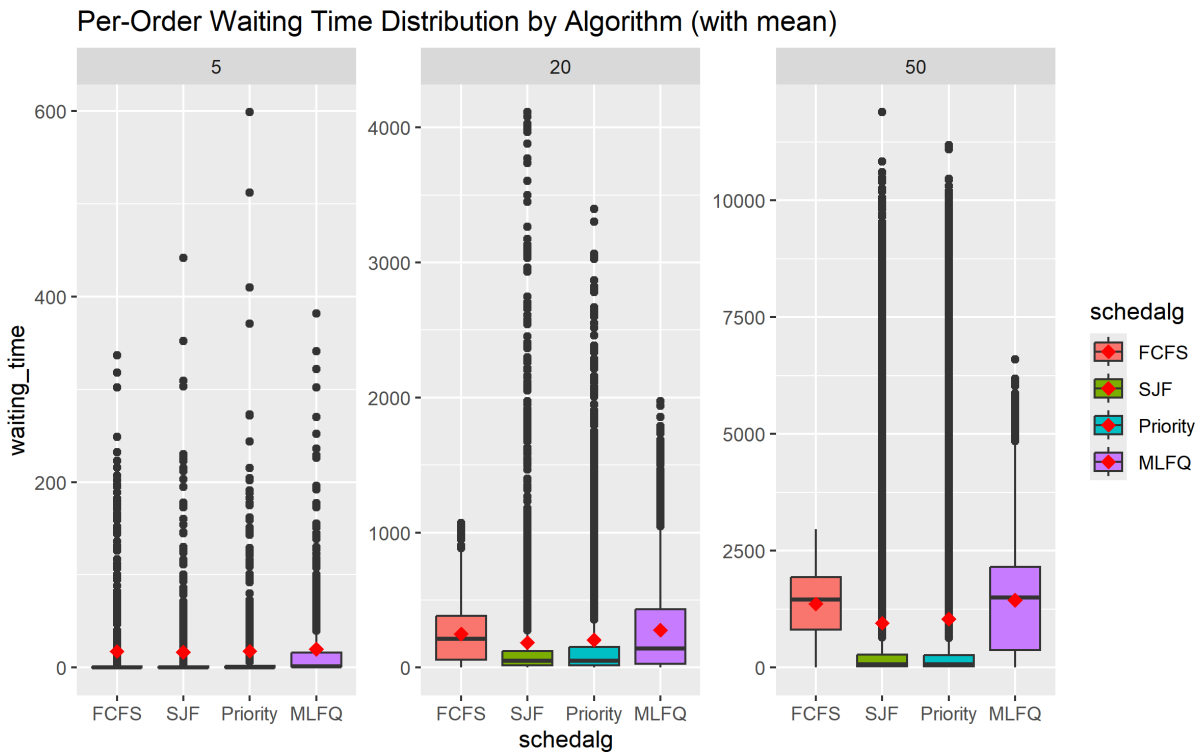

Figure 3: Boxplots of Per-Order Waiting Times

As seen in Figure 3

Per-Patron metrics

What does this mean in terms of the bar and CPU scheduling

Response and waiting time

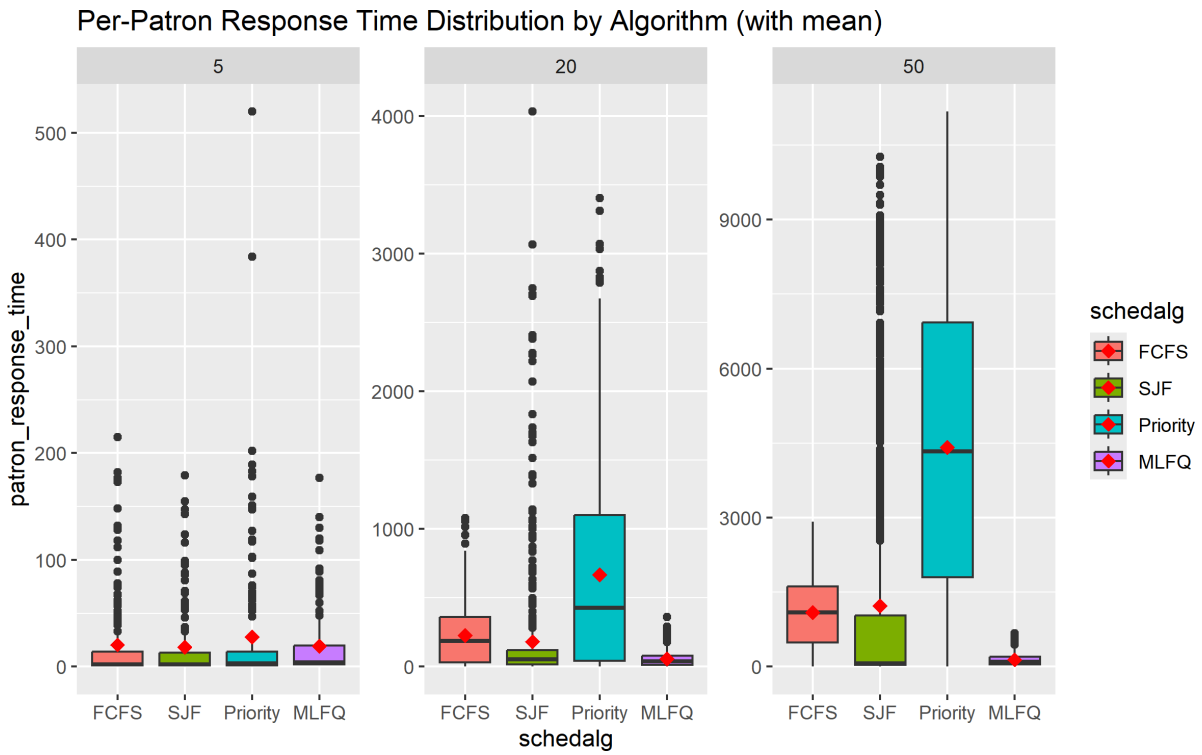

Figure 4: Boxplots of Per-Patron Response Times

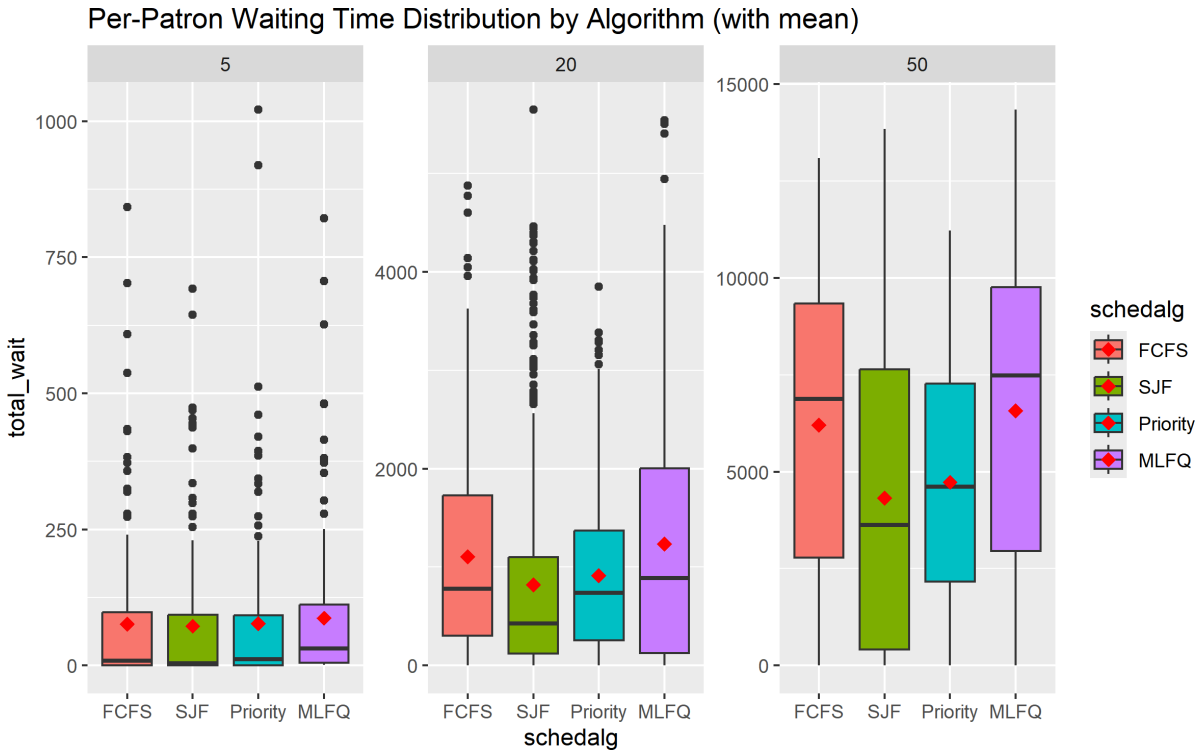

Figure 5: Boxplots of Per-Patron Waiting Times

We now analyse these figures.

Turnaround time

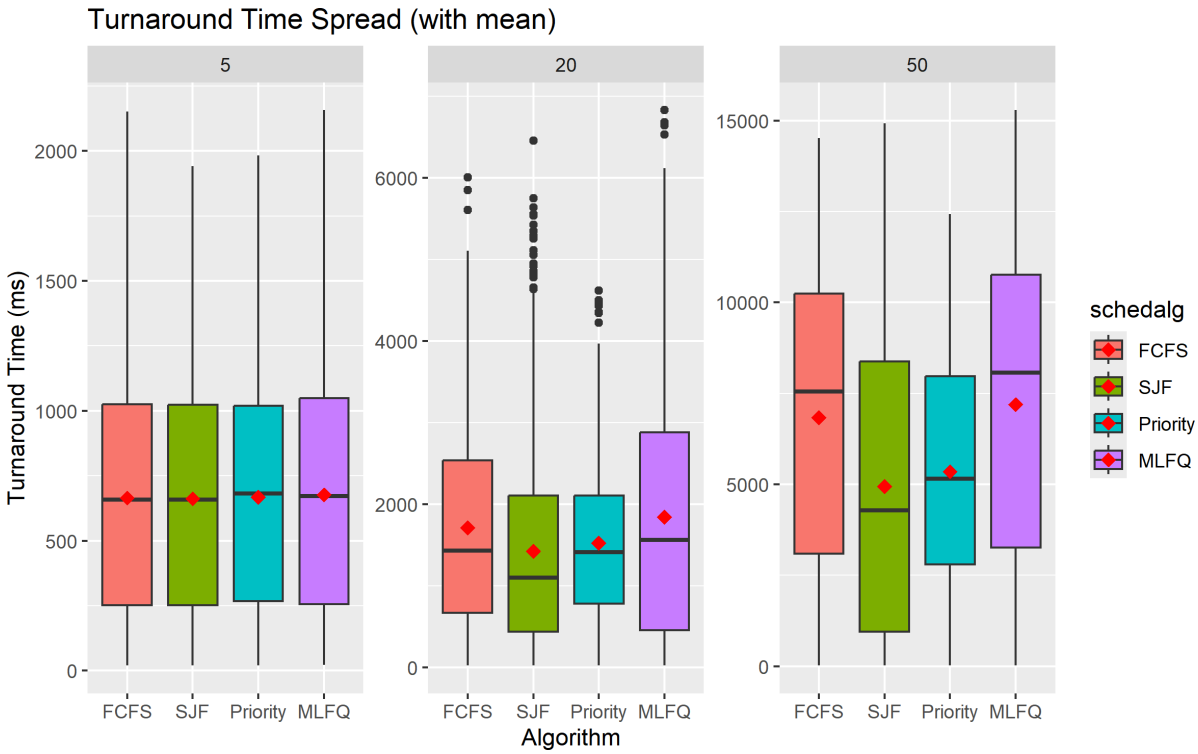

Figure 6: Boxplots of Turnaround Times

The turnaround time.

Throughput

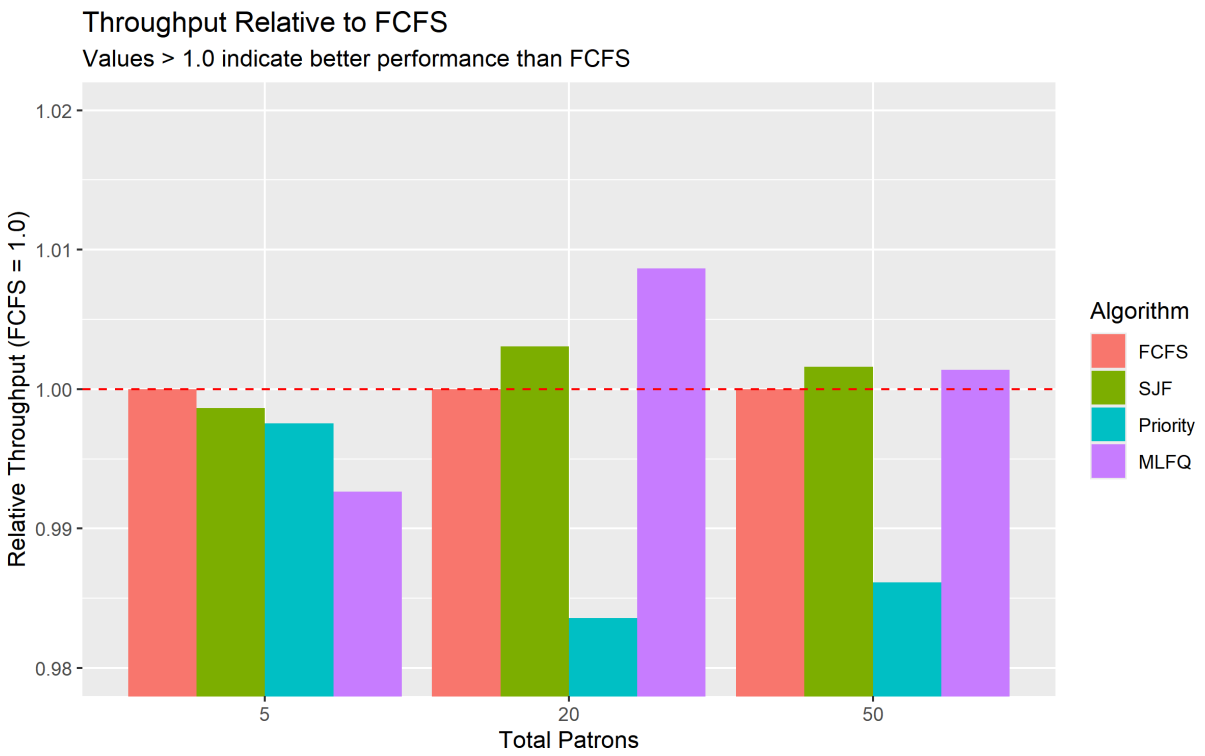

Figure 7: Relative Throughputs compared to FCFS

The throughput results.

Fairness and Predictability

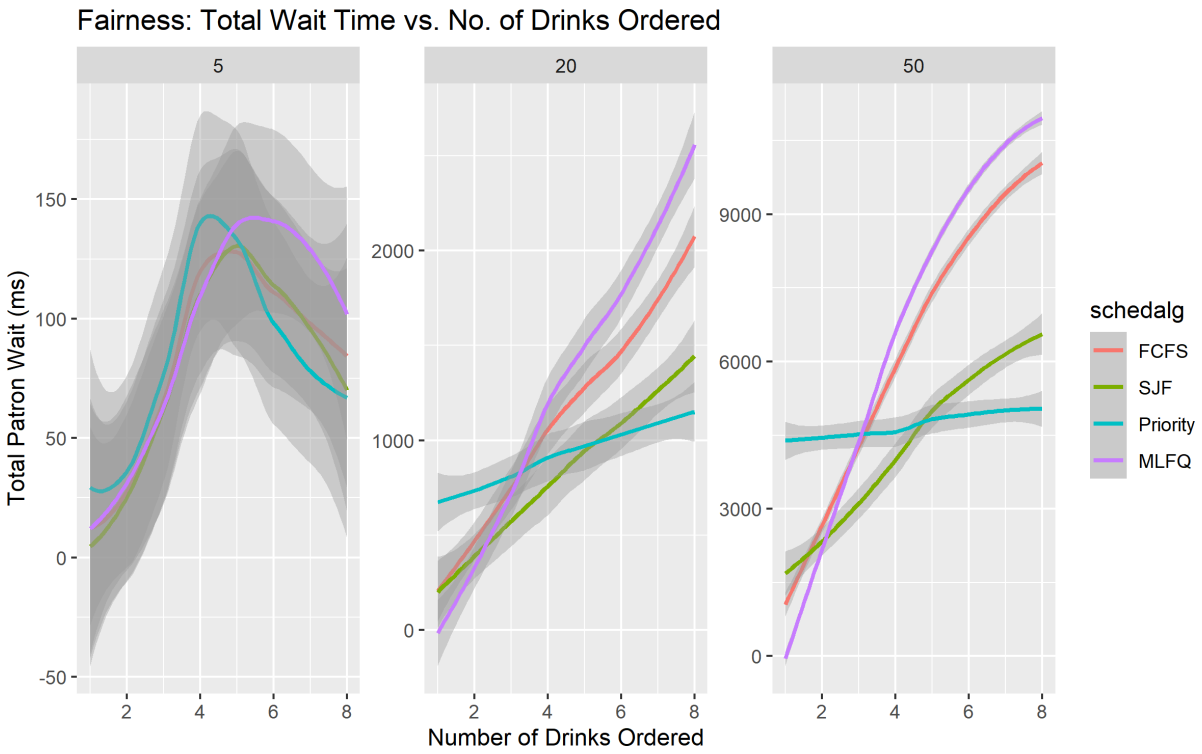

Figure 8: Trendlines of Total Waiting Time vs No. of Drinks Ordered

The fairness of the algorithms.

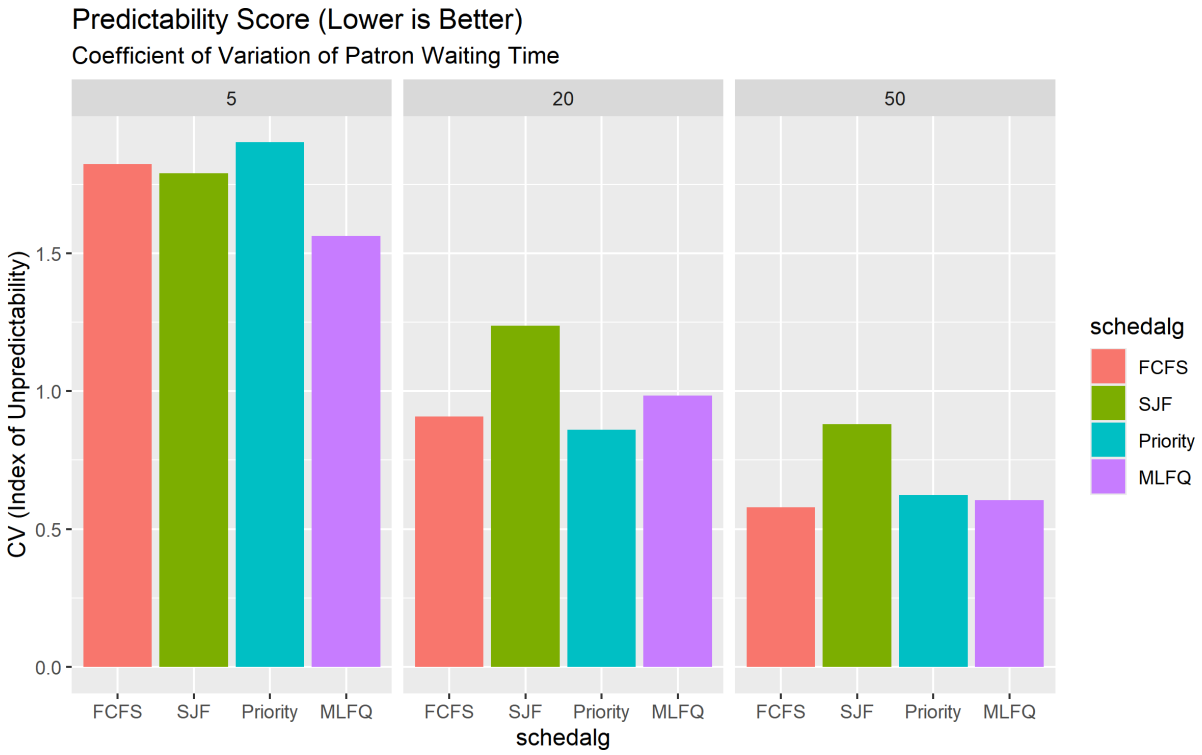

Figure 9: Predictability Score for Patron Waiting Time

Predictability of the algorithms.

Possibility of starvation

As evident in Figure 3.

Conclusions and Recommendations